

SynthLA: Zero-Shot Real-World Robot Control via a Synthetic-Data Language–Action Model

Abstract—Large Language Models (LLMs) and Vision–Language Models (VLMs) have accelerated progress in robot learning, but their deployment typically requires predefined low-level action primitives, large collections of real-world demonstrations, and significant computational resources. To address these limitations, we introduce SynthLA, an 8-billion-parameter GPT-based language–action policy trained entirely on synthetic data using a decoupled vision–reasoning pipeline. In contrast to conventional Vision–Language–Action (VLA) models that operate directly on raw visual input, SynthLA consumes structured textual scene descriptions generated by an upstream perception module. This design enables scalable synthetic-data training, efficient inference (0.33s latency), robot-agnostic execution, and zero-shot deployment in real environments. On four manipulation benchmarks (pick&place, lift, move, and close-the-pot), SynthLA achieves higher success rates than state-of-the-art VLA models while requiring no real-world training data. These results demonstrate that combining synthetic data with modular perception yields a practical and resource-efficient framework for generalizable robot control.

I. INTRODUCTION

Robotic manipulation has advanced significantly in recent years [1], [2], yet a fundamental challenge remains: enabling robots to generalize to novel tasks, unseen objects, and diverse environments [3]. Foundation models such as large language models (LLMs) and vision–language models (VLMs) have shown promise in enhancing robotic reasoning and adaptability [1], [2], [4], [5], [6]. However, their deployment in real-world interaction is limited by difficulties in low-level action execution, reliance on large multimodal datasets, and high computational costs [5], [7], [8]. Collecting large-scale datasets of robot demonstrations, including images and trajectories, is costly and time-consuming [7]. Moreover, training large multimodal models demands substantial GPU resources, which are often unavailable outside dedicated computing centers [8]. These limitations hinder the development of scalable and resource-efficient robot learning systems. Inspired by recent work on modular learning frameworks [9], we propose **SynthLA**, a novel approach for low-level action generation that eliminates the need for real-world data collection. Unlike conventional Vision–Language–Action (VLA) systems that directly consume raw images, SynthLA employs a decoupled pipeline: a dedicated vision module converts scene images into structured textual descriptions, which are then processed by a GPT-based reasoning model. This design offers two main advantages:

- 1) Training data can be synthetically generated at scale;
- 2) Training computational costs are reduced, since the policy model operates on text rather than high-dimensional images.



Fig. 1: Real-world execution of SynthLA on four manipulation tasks: (A) close-the-pot, (B) move, (C) lift, and (D) pick&place.

This modular design makes SynthLA robust to environmental variations such as changes in lighting, background, or workspace surfaces. Unlike VLAs, which typically require recollecting large datasets when task conditions or object sets change, SynthLA only requires re-training the vision module—significantly reducing the time and resources needed. In real-world experiments, SynthLA outperforms state-of-the-art (SOTA) VLA systems [2] on four benchmark manipulation tasks while maintaining real-time performance with an inference latency of ≈ 0.33 seconds. As shown in Section IV-C, smaller GPT variants can further reduce latency with only a modest trade-off in success rate. The contributions of this letter are summarized as follows:

- We present a modular framework for robotic task learning that leverages a GPT-based policy generator decoupled from the vision module, enabling end-to-end training solely on synthetic data;
- We introduce **SynthLA**, an 8-billion-parameter GPT model that (i) achieves zero-shot real-world performance, (ii) operates in real-time with low-latency inference, and (iii) remains robot-agnostic through its vision–text interface;

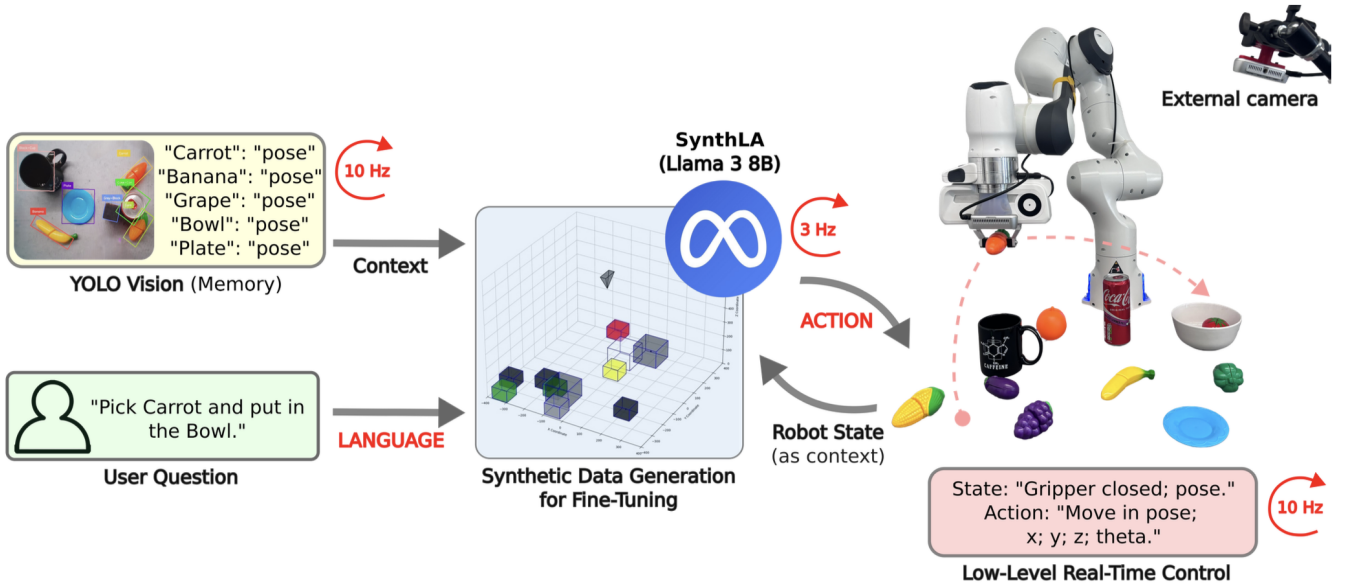


Fig. 2: Overview of SynthLA. The vision module converts scene images into structured textual descriptions, which are combined with the robot state and user instructions. SynthLA then generates the next low-level robot action.

- We demonstrate that SynthLA surpasses SOTA VLA models on four benchmark manipulation tasks, providing a systematic comparison across model scales and task complexity.

II. RELATED WORKS

Language-Guided Task Planning and Multimodal Perception through Vision and Language Models: LLMs are increasingly used for task planning and execution due to their flexibility as frozen modules. Models like SayCan [10] translate natural language instructions into action plans using pre-trained skills, simplifying interaction. Recent advances improve real-world feasibility and resource optimization [11]. LLMs are also extended for long-term planning [12], [13], generating executable code [14], and bridging high-level task planning with low-level control in navigation tasks [15]. Combining LLMs with VLMs enhances decision-making by aligning visual perception with reasoning [16]. For instance, CLIPort [17], Perceiver-Actor [18], and VoxPoser [19] combine VLMs with imitation learning [20], allowing robots to ground their knowledge in the physical space for more contextually aware actions. LLMs and VLMs also enhance reinforcement learning [21] by helping design rewards and interpret tasks [22]. However, these models often lack the physical grounding needed for real-world interaction, excelling more in abstract reasoning.

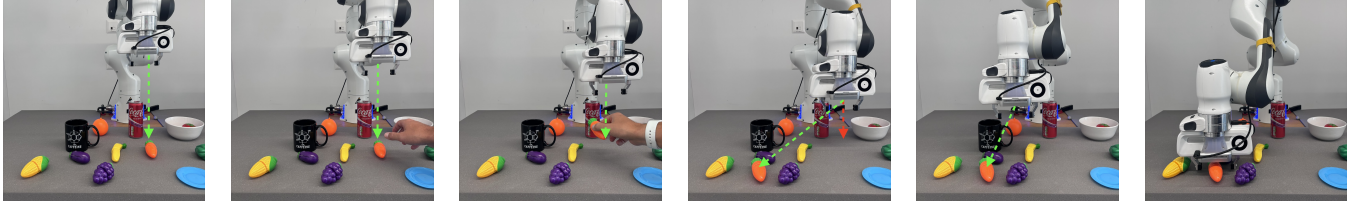
Vision-Language-Action models: Early attempts to integrate LLMs and VLMs in robotics struggled with achieving autonomy and context understanding. Since 2022 [5], the focus has shifted towards developing robotics-specific foundation models using Transformer-based [23] architectures and large-scale domain-specific training. Datasets like Open X-Embodiment [7] and DROID [24] have been key for training in robotic manipulation tasks. New VLA models [2],

[7], [4], [6] take language instructions and visual data to generate control actions. While they aim to leverage in-context learning from their large pre-training to perform real-time control, their high resource demands and slow inference times on common hardware limit their usage in real-world tasks. OpenVLA [2], one of the most advanced VLA models, represents a step forward in terms of adaptation compared to other VLA models that require significantly more challenges for fine-tuning on new tasks [5], [1]. However, it still struggles to work zero-shot with tasks outside its training scope and demands extensive real-world dataset collection, making it resource-intensive to implement [25]. Additionally, most VLAs exhibit non-smooth motions due to imprecise control [26]. Recently, new efforts in developing VLAs have also focused on model size [27] and inference or data efficiency [8], [28].

Unlike these works, SynthLA does not rely on either real or simulated robot trajectories; collecting such data is energy-intensive, time-consuming, and must be repeated whenever tasks or scene elements vary, causing marked performance degradation despite object-level generalization. Whereas VLA models are typically adapted to new environments through supervised fine-tuning with teleoperated demonstrations, our approach eliminates the need for any environment-specific supervision.

III. SYNTHLA: AN EFFICIENT LANGUAGE-ACTION MODEL VIA SYNTHETIC DATA

Our framework addresses the problem of learning a low-latency, language-conditioned policy for robot control across a set of n distinct tasks, using exclusively synthetic data. The method is modular, consisting of (i) a synthetic dataset generator for each task, and (ii) model adaptation for fine-tuning the policy.



(a) SynthLA generates an action to reach the carrot.

(b) Executing the action.

(c) The carrot's position changes.

(d) SynthLA computes a new action.

(e) Approaching the carrot at its new position.

(f) Carrot reached.

Fig. 3: Pick&place task in a dynamic environment. The model is instructed to “pick the carrot and place it in the bowl.” SynthLA initially plans a trajectory (Fig. 3a), but just before execution, the carrot is displaced (Figs. 3b–3d). Within 0.33 s, SynthLA recomputes a trajectory to the new position (Figs. 3e–3f), demonstrating real-time adaptation.

A. Problem Formulation

Let the set of tasks be

$$\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}.$$

Each task τ_k is defined by an instruction space $\mathcal{Q}_k$ (all valid natural-language queries), a state space $\mathcal{S}_k$ (robot–environment configurations), and an action space $\mathcal{A}_k$ (low-level robot commands). We represent a state $s \in \mathcal{S}_k$ as

$$s = (s_{\text{robot}}, s_{\text{env}}),$$

where $s_{\text{robot}} = (\mathbf{p}_{ee}, g)$ contains the end-effector position $\mathbf{p}_{ee} \in \mathbb{R}^3$ and the gripper state $g \in \mathcal{G}$ (binary variable for open or closed). The environment state $s_{\text{env}} = \{(c_i, \mathbf{p}_i, \theta_i)\}_{i=1}^M$ describes M objects, where c_i is the object class, $\mathbf{p}_i \in \mathbb{R}^3$ its position, and θ_i its yaw orientation about the vertical axis. A multi-task policy is defined as

$$\pi : \mathcal{Q} \times \mathcal{S} \rightarrow \mathcal{A},$$

mapping a query–state pair (q, s) to the next low-level action, where $\mathcal{Q} = \bigcup_{k=1}^n \mathcal{Q}_k$, $\mathcal{S} = \bigcup_{k=1}^n \mathcal{S}_k$, and $\mathcal{A} = \bigcup_{k=1}^n \mathcal{A}_k$. In practice, π may be trained jointly over all tasks to produce a single multi-task policy, or separately on $(\mathcal{Q}_k, \mathcal{S}_k, \mathcal{A}_k)$ to yield task-specific policies π_k .

B. Synthetic Dataset Generation

Several large-scale robotics datasets exist for grasping, demonstrations, and interaction learning [29], [30], [31]. While some, such as RoboNet [32] and Open X-Embodiment [7], span multiple robot platforms, most focus on a single robot. These datasets have advanced robot learning but are costly to produce, requiring extensive human involvement and real-world image collection, which makes them resource-intensive and difficult to scale. In contrast, SynthLA does not require image-based datasets. Instead, it uses textual descriptions of scenes, enabling all data to be generated in simulation without the need for real images. For each task $\tau_k \in \mathcal{T}$, we define a generator

$$G_k : \mathbb{Z}^+ \rightarrow \mathcal{Q}_k \times \mathcal{S}_k \times \mathcal{A}_k,$$

which produces (q, s, a^*) , where q is a valid query, s is a randomized symbolic state, and $a^* = \pi_k(q, s)$ is the

correct low-level action. The generator is composed of three components: *Task Decomposition*, *Scene Randomization*, and a *Deterministic Action Oracle*.

Task Decomposition. Each task τ_k is decomposed into an ordered sequence of L_k sub-tasks:

$$\tau_k = \{\sigma_1, \sigma_2, \dots, \sigma_{L_k}\},$$

where each σ_j corresponds to a parametrized low-level sub-task required to complete the task.

Scene Randomization. At the beginning of each episode, the initial state is randomized as

$$s^{(0)} \sim \mathcal{P}_{\text{init}},$$

where $\mathcal{P}_{\text{init}}$ is a distribution over feasible robot and object configurations (end-effector pose, object positions, and orientations).

Deterministic Action Oracle. Given a state $s^{(j-1)}$ and sub-task σ_j , the oracle deterministically returns the correct action:

$$a_j^* = \mathcal{O}(s^{(j-1)}, \sigma_j).$$

The scene is then updated by simulating this action:

$$s^{(j)} = \Gamma(s^{(j-1)}, a_j^*),$$

where Γ denotes the environment transition function. Each dataset entry is thus a triplet

$$(q_j, s^{(j-1)}, a_j^*),$$

where $q_j \in \mathcal{Q}_k$ is the natural-language instruction for task τ_j . Sequentially, the generator produces a trajectory of such entries for all sub-tasks. The complete dataset for task τ_k is obtained by aggregating over N_k randomized episodes:

$$\mathcal{D}_k = \bigcup_{i=1}^{N_k} \bigcup_{j=1}^{L_k} \left\{ (q_j^{(i)}, s^{(j-1,i)}, a_j^{*(i)}) \right\}.$$

The full multi-task dataset is then

$$\mathcal{D} = \bigcup_{k=1}^n \mathcal{D}_k.$$

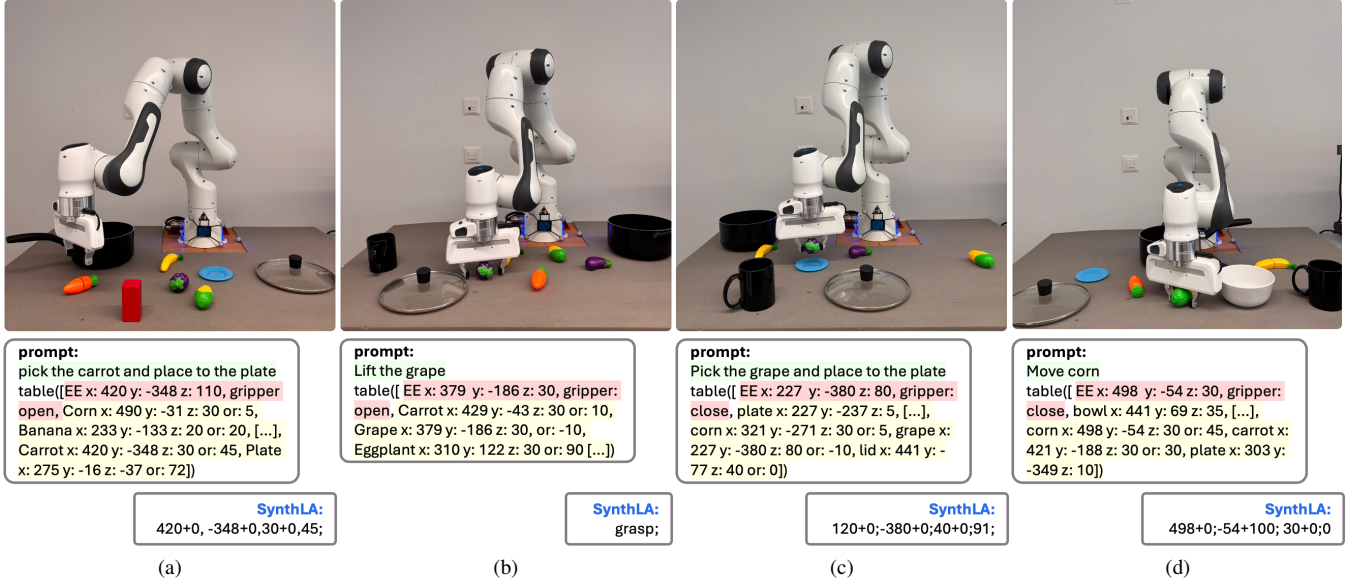


Fig. 4: Real-world experiment with SynthLA QAs, where the prompt encodes the user request (green), robot state (red), and scene objects with positions (yellow).

C. Model Adaptation

We initialize with a pre-trained GPT-based policy $f_{\theta_0} : \mathcal{Q} \times \mathcal{S} \rightarrow \mathcal{A}$. This policy is fine-tuned on the synthetic dataset $\mathcal{D}$ (2.4M triplets) using *QLoRA* [33], a parameter-efficient fine-tuning method. QLoRA quantizes the base model weights into low-bit precision and introduces trainable low-rank adapters, which capture task-specific knowledge while leaving the original weights frozen.

Each weight matrix $W \in \mathbb{R}^{m \times n}$ is approximated as

$$W \approx W_q + \Delta W, \quad \Delta W = BA^\top,$$

where W_q is quantized and frozen, and $A \in \mathbb{R}^{n \times r}$, $B \in \mathbb{R}^{m \times r}$ are trainable low-rank matrices ($r \ll \min(m, n)$). The adapted policy is thus

$$f_{\theta}(q, s) = f_{\theta_0}(q, s; W_q + BA^\top).$$

In SynthLA, the backbone f_{θ_0} is Llama 3.1 (8B parameters) [34]. Deployed on a single NVIDIA RTX 4090, the system achieves an average latency of ≈ 0.33 seconds per action (≈ 3 Hz), supporting real-time robot control (Fig. 3).

D. Perception

At deployment, a perception module $\phi : \mathcal{O} \rightarrow s_{\text{env}}$ maps raw observations $o \in \mathcal{O}$ (e.g., RGB-D images) into a structured symbolic state. At each time step t , one or multiple cameras capture observations, which are stored in a shared memory system to mitigate occlusions. For each camera $c \in \mathcal{C}$, the perception model ϕ_c processes the observation $o_t^{(c)}$. Its output is

$$\phi_c(o_t^{(c)}) = \text{obj}_{\text{px}}^{(c)} = \{(c_i, \mathbf{px}_i, d_i, \alpha_i)\}_{i=1}^M,$$

where c_i is the object class, $\mathbf{px}_i \in \mathbb{R}^2$ is the 2D pixel location, $d_i \in \mathbb{R}$ is the depth value, and α_i is the orientation

in the image frame. Using intrinsic and extrinsic calibration, this is transformed into a 3D environment state estimate:

$$s_{\text{env}}^{(c)} = \{(c_i, \mathbf{p}_i, \theta_i)\}_{i=1}^M,$$

where $\mathbf{p}_i \in \mathbb{R}^3$ is the object's position in world coordinates. A memory module aggregates all $s_{\text{env}}^{(c)}$ into a unified s_{env} . In practice, given the limited number of objects, a single Intel RealSense camera [35] mounted externally was sufficient. The perception module uses a YOLO OBB v8 model fine-tuned on a custom dataset. Inference runs at 10 Hz on RTX 4090, ensuring real-time operation.

IV. EXPERIMENTS

We evaluate SynthLA in real-world scenarios under two conditions. In the *static* setting, the scene remains unchanged during inference. In the *dynamic* setting, objects may be displaced or new elements introduced during task execution. For both conditions, we consider two data regimes: *in-distribution* (ID), where all objects were present in the fine-tuning dataset, and *out-of-distribution* (OOD), where the model encounters novel objects not present in the training dataset. The experimental evaluation addresses the following research questions:

- 1) How does SynthLA compare with a SOTA VLA model across benchmark tasks? In particular, how does performance differ between ID and OOD conditions?
- 2) How does performance scale with SynthLA's model size?
- 3) How does SynthLA's performance on a specific task scale with the number of tasks it has been trained on?

TABLE I: QUANTITATIVE RESULTS OF COMPARISON BETWEEN SYNTHLA AND OPENVLA

Task	static environment (%)							dynamic environment (%)						
	in-distribution $\uparrow$				out-of-distribution $\uparrow$			in-distribution $\uparrow$				out-of-distribution $\uparrow$		
	lift	pick&place	move	close-the-pot	lift	pick&place	move	lift	pick&place	move	close-the-pot	lift	pick&place	move
OpenVLA	0	0	0	0	0	0	0	0	0	0	0	0	7	0
OpenVLA150	13	20	13	13	7	13	7	13	13	13	7	7	7	7
OpenVLA300	33	33	40	20	20	20	13	13	13	13	7	13	13	13
OpenVLA450	67	67	60	47	47	40	47	40	40	47	33	20	13	20
SynthLA	80	73	80	60	73	67	73	73	60	73	47	60	53	67

To evaluate the performance of SynthLA relative to standard VLA models, we use SOTA OpenVLA [2] as the baseline for comparison. This choice is motivated by two factors: (i) the lack of LLM-based architectures in the literature with capabilities comparable to SynthLA, and (ii) both models belonging to the GPT family with similar inference times.

Tasks: Four common tasks were considered for evaluation: *lift*, *pick&place*, *move*, and *close-the-pot*.

Training setup: SynthLA, which is fine-tuned exclusively on synthetic data, is deployed in the real world in a zero-shot fashion. The synthetic dataset consists of 2.4 million samples, evenly distributed across the four tasks (500,000 per task). Fine-tuning was performed for a single epoch on an NVIDIA RTX 4090 GPU, requiring approximately 16 hours. In contrast, OpenVLA requires fine-tuning using real-world demonstrations. These demonstrations were collected by teleoperating the robot using a SpaceMouse. Each sample contained the camera image, user instructions, and robot pose. In total, 450 demonstrations per task were recorded (1,800 in total, ~ 60 hours).

Evaluation protocol: The performance of each model was comprehensively evaluated across all tasks, with success rate as the primary metric. The Success Rate (%) is defined as the proportion of experiments in which the model successfully completed the task, relative to the total number of trials for that task. Each model was evaluated across all tasks and conditions, with 15 trials conducted under each condition for each task.

SynthLA vs OpenVLA: To assess the performance of SynthLA, we evaluated four OpenVLA variants: **Zero-shot** (no fine-tuning); **OpenVLA150** (150 demonstrations per task); **OpenVLA300** (300 demonstrations per task); and **OpenVLA450** (450 demonstrations per task). Initial 8-bit quantized training yielded poor results, so we retrained without quantization on an RTX 5090 (32 GB). Training times were: 26 h (OpenVLA150), 36 h (OpenVLA300), and 40 h (OpenVLA450).

A. Quantitative Results

1) *Static environment, ID and OOD:* Table I (left) reports static scene success rates for SynthLA and the OpenVLA variants. The results show that SynthLA achieves high ID and OOD performance across all tasks without any real-world data, consistently outperforming the OpenVLA variants fine-

tuned with hundreds of demonstrations. The OOD results highlight that SynthLA retains a success rate on new objects, whereas the performance of OpenVLA degrades markedly, highlighting its limited ability to generalize. For both models, the task exhibiting the lowest success rate is the close-the-pot task, which is particularly demanding as it requires precise control for both grasping the lid and accurately aligning it with the pot. An additional source of failure originates from the depth sensor, which frequently fails to reliably estimate the pot’s position due to strong surface reflectance. This limitation was partially alleviated by applying a matte adhesive coating to the pot surface.

2) *Dynamic environment, ID and OOD:* To test online robustness, we repeated evaluations while perturbing object positions during execution (see Fig. 3). The results in Table I (right) indicate that SynthLA achieves robust behavior in dynamic environments, recomputing actions rapidly and adapting to the objects’ displacement during execution. OpenVLA, in contrast, exhibits unpredictable behavioral changes when the operator’s hand enters the scene to reposition the objects. This effect is likely due to the absence of external elements, such as hands, in the dataset used for fine-tuning. Because SynthLA relies on structured scene representation rather than raw images, perception can filter out irrelevant visual clutter. This reduces unnecessary input variations reaching the model and helps stabilize performance under a dynamic environment. These results highlight that the modular pipeline employed in SynthLA can be more robust in dynamic, partially observable scenes, instead of end-to-end pixel-to-action VLA’s even when trained on large demonstration datasets.

B. Qualitative Results

Figure 4 shows representative real-robot rollouts and model outputs. In Figure 4a, corresponding to the “*pick the carrot and place on the plate*” task, the end-effector (EE) is positioned above the carrot, and SynthLA suggests moving closer to the object. Figure 4b illustrates the “*lift the grape*” task, where the EE is nearly aligned with the grape and SynthLA recommends closing the gripper. In Figure 4c, during the “*pick the grape and place on the plate*” task, SynthLA suggests opening the gripper, consistent with the object release phase. Finally, Figure 4d shows the “*move*

TABLE II: SYNTHLA SCALING COMPARISON

Task	inf. time [ms]↓	fine-tuning time [h]↓	#GPU↓	in-distribution (%)↑				out-of-distribution (%)↑		
				lift	pick&place	move	close-the-pot	lift	pick&place	move
1B	0.10	2	1	13	13	7	7	7	7	7
3B	0.19	7	1	27	33	27	40	20	27	20
8B	0.33	16	1	80	73	80	60	73	67	73
70B	1.62	48	8	87	80	87	80	80	73	80

corn” task, where the EE is already grasping the object and SynthLA recommends moving the EE. Selected experiments were recorded and are available online¹ as supplementary material.

TABLE III: IMPACT SKILL COUNT ON PICK&PLACE

SynthLA	Pick&place (%) ↑	
	In-distribution	Out-of-distribution
1-skill	87	80
2-skills	80	73
3-skills	80	73
4-skills	73	67

C. Ablation Studies

1) *Scaling with model parameters*: We evaluate four models from the Llama family: Llama 3.2 1B, Llama 3.2 3B, Llama 3.1 8B, and Llama 3.3 70B. Each was fine-tuned using QLoRA under identical conditions (dataset, epochs, and hardware) except for the 70B model, which has been 2-bit quantized for resource constraints. Table II reports success rates (ID and OOD), fine-tuning time, inference latency, and GPU minimum requirements for fine-tuning. Performance scales positively with model size. The 70B model achieves the highest success rate but is impractical due to extreme training cost and inference demands. In contrast, the 8B model offers the best trade-off between accuracy and efficiency, achieving strong performance with manageable computational requirements.

2) *Scaling with number of skills*: We further evaluate SynthLA (8B model) when trained on progressively larger task repertoires. Specifically, we investigate how task diversity influences the model’s performance on the *pick&place* task by fine-tuning the model under four regimes: (i) one skill (*pick&place* only), (ii) two skills (*pick&place* + *lift*), (iii) three skills (adding *move*), and (iv) four skills (full set). Each task was allocated an equal number of demonstrations to ensure comparability. Table III reports the success rate of *pick&place* on ID and OOD under a static environment. As the number of tasks increases, the success rate of *pick&place* drops moderately (from 87% with 1 skill to 73% with 4 skills in ID). This illustrates a trade-off: broader training improves cross-task generalization but can marginally degrade single-task performance, since the limited model capacity must be distributed across tasks.

V. LIMITATIONS

Despite its advantages, SynthLA presents several limitations. First, constructing the synthetic dataset requires a careful task analysis and decomposition into parameterized sub-tasks. This process demands nontrivial engineering effort and domain expertise. However, once the pipeline is defined, it can generate millions of training samples within seconds, which mitigates the upfront cost. Second, the current vision component constrains generalization. The framework fine-tunes an object detector on a predefined set of objects, limiting recognition of unseen categories. Vision Transformers (ViTs), with their broader recognition capabilities enabled by large-scale pretraining, represent a promising direction to address this limitation. Third, the inference speed of large language models remains a bottleneck. The 8B-parameter model used in this work operates at approximately 3 Hz, which is sufficient for the tasks considered but inadequate for faster or more dynamic scenarios. Future work will explore smaller models to improve control frequency while preserving task success rates. Finally, while the tasks presented may appear relatively simple, a more advanced version of SynthLA is under development to address increasingly complex tasks. It is also important to note that the proposed model has been trained exclusively on synthetic data, with no dataset collected from real-world environments. Nonetheless, the framework is immediately deployable across diverse contexts and conditions.

VI. CONCLUSION

In this letter, we presented **SynthLA**, a modular language-action policy trained exclusively on synthetic data and deployed zero-shot in real robotic environments. By decoupling perception from reasoning, SynthLA leverages structured scene descriptions instead of raw visual inputs, enabling scalable dataset generation, efficient fine-tuning, and real-time inference on consumer-grade hardware. Extensive evaluations across four manipulation benchmarks demonstrated that SynthLA achieves higher success rates than SOTA VLA models while requiring no real-world demonstrations, highlighting its strong generalization to out-of-distribution objects and robustness under dynamic scene changes. Beyond its empirical performance, the results provide evidence that synthetic training combined with modular vision-text interfaces can overcome key limitations of current VLA approaches, such as data collection costs and hardware inefficiency. Future work will focus on extending

¹<https://synthla.github.io>

SynthLA to more diverse task repertoires, improving multi-task scaling, and integrating additional sensing modalities. We believe this paradigm offers a promising path toward practical, resource-efficient, and generalizable robot control systems.

REFERENCES

- [1] A. Brohan et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [2] M. J. Kim et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [3] P. Intelligence et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025.
- [4] Octo Model Team et al. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024.
- [5] A. Brohan et al. Rt-1: Robotics transformer for real-world control at scale. *Proceedings of Robotics: Science and Systems*, 2022.
- [6] K. Black et al. pi0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] A. Padalkar et al. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023.
- [8] J. Wen et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2409.12514*, 2024.
- [9] R. A. Izzo et al. Btgenbot: Behavior tree generation for robotic tasks with lightweight llms. *ArXiv*, abs/2403.12761, 2024.
- [10] M. Ahn et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [11] R. Hazra et al. Saycanpay: Heuristic planning with large language models using learnable domain knowledge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38(18), pp. 20123–20133, 2024.
- [12] K. Lin et al. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots*, 47(8):1345–1365, 2023.
- [13] W. Huang et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- [14] I. Singh et al. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11523–11530. IEEE, 2023.
- [15] D. Shah et al. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action. In *Conference on robot learning*, pp. 492–504. PMLR, 2023.
- [16] D. Driess et al. Palm-e: An embodied multimodal language model. In *arXiv preprint arXiv:2303.03378*, 2023.
- [17] M. Shridhar et al. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pp. 894–906. PMLR, 2022.
- [18] M. Shridhar et al. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.
- [19] W. Huang et al. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.
- [20] S. Sontakke et al. Roboclip: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems*, 36, 2024.
- [21] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, 2018.
- [22] Y. Wang et al. Rl-rlm-f: Reinforcement learning from vision language foundation model feedback. In *Proceedings of the 41th International Conference on Machine Learning*, 2024.
- [23] A. Vaswani et al. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [24] A. Khazatsky et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [25] X. Li et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.
- [26] S. Zhang et al. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. *arXiv preprint arXiv:2412.18194*, 2024.
- [27] S. Belkhale and D. Sadigh. Minivla: A better vla with a smaller footprint, 2024.
- [28] M. J. Kim et al. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [29] F. Ebert et al. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. *CoRR*, abs/1812.00568, 2018.
- [30] X. Zhu et al. Fanuc manipulation: A dataset for learning-based manipulation with fanuc mate 200id robot. <https://sites.google.com/berkeley.edu/fanuc-manipulation>, 2023.
- [31] D. Kalashnikov et al. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *CoRR*, abs/2104.08212, 2021.
- [32] S. Dasari et al. Robonet: Large-scale multi-robot learning. *arXiv preprint arXiv:1910.11215*, 2019.
- [33] T. Dettmers et al. Qlora: Efficient finetuning of quantized llms. In A. Oh et al., editors, *Advances in Neural Information Processing Systems*, volume 36, pp. 10088–10115. Curran Associates, Inc., 2023.
- [34] A. Dubey et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [35] L. Keselman et al. Intel realsense stereoscopic depth cameras. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pp. 1–10, 2017.